

AI – Assisted Coding

END LAB TEST

Name: T.Sai Varun

Roll.no: 2403A510C0

Batch: 05

Date: 24 – 11 – 2025

SET – 2: Ethical AI Practices in Student Data Handling

Q1: Protect student PII in LMS logs

- **Task 1:** Use AI to identify possible privacy violations in a given code snippet.
- **Task 2:** Modify the snippet using AI suggestions to implement masking or hashing.

Q2: Bias mitigation in recommendation engine

- **Task 1:** Ask AI to detect biased logic in course recommendation rules.
- **Task 2:** Refactor code with fairness constraints

Q1. PROMPT:

Task – 1:

“Analyze this LMS logging code for PII exposure and privacy violations:

[ENTER SNIPPET]

Identify what student data is exposed and shouldn't be logged in plain text.”

Task – 2:

“Refactor this code to mask/hash all student PII using appropriate methods:

[ENTER SNIPPET]

Use SHA-256 for hashing and partial masking where needed. Add comments.”

CODE:

Task – 1:

```
task1.html > ...
1  <!doctype html>
2  <html lang="en">
3  <head>
4  <meta charset="utf-8" />
5  <title>LMS Submit – Safe Logging Demo</title>
6  <meta name="viewport" content="width=device-width,initial-scale=1" />
7  <style>
8  :root { --bg: #0f172a; --card: #0b1220; --muted: #94a3b8; --accent: #60a5fa; --ok: #16a34a; }
9  body{font-family:Inter,system-ui,Segoe UI,Arial;background:linear-gradient(180deg,#071024,#07132a);color:#e6eef8;margin:0;padding:24px;}
10 .wrap{max-width:980px;margin:0 auto;display:grid;grid-template-columns:1fr 380px;gap:20px;}
11 .card{background:linear-gradient(180deg,#255,255,0.02),#255,255,0.01);padding:18px;border-radius:12px;box-shadow:0 6px 24px #255,255,0.6);}
12 h1{margin:0 0 12px;font-size:20px}
13 label{display:block;margin:8px 0 4px;color:var(--muted);font-size:13px}
14 input[type=“text“], input[type=“file“, select {width:100%;padding:8px;border-radius:8px;border:1px solid #255,255,255,0.05);background:transparent;color:inherit}
15 .row{display:flex;gap:12px}
16 .small{flex:1}
17 button{background:var(--accent);color:#022;border:0;padding:10px 14px;border-radius:10px;font-weight:600;cursor:pointer;margin-top:12px}
18 .muted{color:var(--muted);font-size:13px;margin-top:6px}
19 .log{height:380px;overflow:auto;background:#020617;border-radius:10px;padding:12px;font-family:monospace;font-size:13px;color:#bcd;white-space:pre-wrap}
20 .controls{display:flex;gap:8px;align-items:center;margin-top:8px}
21 .chip{background:#07162b;padding:6px 10px;border-radius:999px;font-size:12px;color:var(--muted)}
22 .img-preview{max-width:100%;border-radius:8px;border:1px solid #255,255,255,0.03);margin-top:10px}
23 footer{grid-column:1/-1;margin-top:12px;color:var(--muted);font-size:13px}
24 .toggle{display:flex;align-items:center;gap:8px}
25 .mode {font-weight:600;color:var(--muted)}
26 </style>
27 </head>
28 <body>
29 <div class=“wrap“>
30 <div class=“card“>
31 <h1>Submit Assignment – Safe logging demo</h1>
32
33 <form id=“frm“ onsubmit=“return handleSubmit(event)“>
34 <label>user_id</label>
35 <input id=“user_id“ type=“text“ value=“stud23“ />
36
37 <label>name</label>
38 <input id=“name“ type=“text“ value=“Akshitha“ />
39
40 <label>email</label>
41 <input id=“email“ type=“text“ value=“akshitha@example.com“ />
42
43 <div class=“row“>
44 <div class=“small“>
45 <label>course_id</label>
```

```

46     <input id="course_id" type="text" value="CS101" />
47   </div>
48   <div class="small">
49     <label>assignment</label>
50     <input id="assignment" type="text" value="hw1" />
51   </div>
52 </div>
53
54 <label>grade</label>
55 <input id="grade" type="text" value="A" />
56
57 <label>file</label>
58 <input id="file" type="file" />
59
60 <div class="controls">
61   <label class="toggle"><input id="use_server" type="checkbox" /> <span class="muted">Send to real server</span></label>
62   <div class="chip" id="endpoint">Endpoint: <code>http://127.0.0.1:5000/submit_assignment</code></div>
63 </div>
64
65 <button type="submit">Submit</button>
66 <div class="muted">Mode: <span id="mode" class="mode">Standalone (simulate)</span></div>
67 </form>
68
69 <div style="margin-top:14px">
70   <button onclick="downloadLogs()">Download logs</button>
71   <button onclick="clearLogs()" style="margin-left:8px">Clear logs</button>
72 </div>
73
74 <div style="margin-top:12px">
75   <strong>Example screenshot from server path</strong>
76   
77   <div class="muted">(If your environment exposes files under /mnt/data this image will show.)</div>
78 </div>
79 </div>
80
81 <div class="card">
82   <h1 style="font-size:16px;margin-bottom:8px">Log Console</h1>
83   <div id="log" class="log" aria-live="polite"></div>
84   <div style="margin-top:8px;color:var(--muted);font-size:13px">
85     These logs are <strong>safe</strong> (user id is pseudonymized, email is redacted, full grade not logged).
86
87 </div>
88
89 <footer>
90   Tip: open devtools (F12) to see network requests. To send to your Flask server, enable "Send to real server" and ensure Flask is running at the endpoint shown above.
91 </footer>
92 </div>
93
94 <script>
95   // ----- helpers -----
96   const logEl = document.getElementById('log');
97   function appendLog(...lines){
98     const ts = new Date().toLocaleString();
99     logEl.textContent = (logEl.textContent ? logEl.textContent + "\\n" : "") + lines.join(' ') + "\\n";
100    logEl.scrollTop = logEl.scrollHeight;
101  }
102  function clearLogs(){ logEl.textContent=''; appendLog('[console cleared]'); }
103  function downloadLogs(){
104    const blob = new Blob([logEl.textContent], {type:'text/plain;charset=utf-8'});
105    const url = URL.createObjectURL(blob);
106    const a = document.createElement('a');
107    a.href = url; a.download = 'lms_safe_logs.txt'; a.click();
108    URL.revokeObjectURL(url);
109  }
110
111  // simple email redact
112  Complexity is 4 Everything is cool!
113  function redactEmail(email){
114    if(!email) return 'unknown';
115    return email.replace(/^(.+)(@.+)$/, '$1***$2');
116  }
117
118  // pseudonymize (sha-256) -> first 8 hex chars
119  Complexity is 5 Everything is cool!
120  async function pseudonymize(val){
121    if(!val) return 'unknown';
122    const enc = new TextEncoder().encode(val);
123    const hash = await crypto.subtle.digest('SHA-256', enc);
124    const hex = Array.from(new Uint8Array(hash)).map(b => b.toString(16).padStart(2,'0')).join('');
125    return hex.slice(0,8);

```

```

124     }
125
126     // bucket grade (we don't log exact)
127     Complexity is 10 it's time to do something...
128     function gradeBucket(g){
129         if(!g) return 'none';
130         const val = String(g).trim().toUpperCase();
131         if(['A+', 'A'].includes(val)) return 'A';
132         if(['B+', 'B'].includes(val)) return 'B';
133         if(['C', 'C+', 'D', 'F'].includes(val)) return 'C_or_lower';
134         return 'other';
135     }
136
137     // ----- submit handler -----
138     const form = document.getElementById('frm');
139     const useServerCheckbox = document.getElementById('use_server');
140     const modeSpan = document.getElementById('mode');
141
142     useServerCheckbox.addEventListener('change', () => {
143         modeSpan.textContent = useServerCheckbox.checked ? 'Server mode (will POST to endpoint)' : 'Standalone (simulate)';
144     });
145
146     Complexity is 10 it's time to do something...
147     async function handleSubmit(e){
148         e.preventDefault();
149         const user_id = document.getElementById('user_id').value;
150         const name = document.getElementById('name').value;
151         const email = document.getElementById('email').value;
152         const course_id = document.getElementById('course_id').value;
153         const assignment = document.getElementById('assignment').value;
154         const grade = document.getElementById('grade').value;
155         const fileInput = document.getElementById('file');
156         const file = fileInput.files[0];
157
158         const pid = await pseudonymize(user_id);
159         const emailMasked = redactEmail(email);
160         const fileName = file ? file.name : 'no-file';
161         const ip = 'client-side'; // we can't get remote IP from browser reliably
162
163         // safe logging - do not log raw identifiers
164         appendLog(`[INFO] Submission received: pid=${pid} course=${course_id} assignment=${assignment} file=${fileName} ip=${ip}`);
165
166         if(grade) appendLog(`[INFO] Grade provided for pid=${pid} (presence logged, not exact)`);
167
168         // If server mode, do a real POST to your Flask server endpoint (multipart/form-data)
169         if(useServerCheckbox.checked){
170             try {
171                 appendLog(`[INFO] Sending to server endpoint...`);
172                 const endpoint = 'http://127.0.0.1:5000/submit_assignment';
173                 const fd = new FormData();
174                 fd.append('user_id', user_id);
175                 fd.append('name', name);
176                 fd.append('email', email);
177                 fd.append('course_id', course_id);
178                 fd.append('assignment', assignment);
179                 fd.append('grade', grade);
180                 if(file) fd.append('file', file, fileName);
181
182                 const res = await fetch(endpoint, { method: 'POST', body: fd });
183                 const data = await res.json().catch(()=>({}));
184                 appendLog(`[SERVER] HTTP ${res.status} ${res.statusText} - response: ${JSON.stringify(data)}`);
185             } catch(err){
186                 appendLog(`[ERROR] Failed to send to server: ${err.message || err}`);
187                 console.error(err);
188             }
189         } else {
190             // simulate server processing locally (no sensitive data stored)
191             const simulatedResponse = { status: 'ok', pid };
192             appendLog(`[SIM] Processing done - response: ${JSON.stringify(simulatedResponse)}`);
193         }
194
195         return false;
196     }
197
198     // init
199     clearLogs();
200     appendLog(`[app] Safe logging frontend ready. Use "Send to real server" to POST to Flask at http://127.0.0.1:5000/submit_assignment`);
201
202     </script>
203 </body>
204 </html>
205

```

Task – 2:

```

1 <!doctype html>
2 <html lang="en">
3 <head>
4 <meta charset="utf-8" />
5 <meta name="viewport" content="width=device-width,initial-scale=1" />
6 <title>LMS Safe Logging – Frontend Demo</title>
7 <style>
8   body{font-family:Inter,system-ui,Arial;background:linear-gradient(to right, #000000 0%, #000000 49%, #000000 49%, #000000 51%, #000000 51%, #000000 53%, #000000 53%, #000000 55%, #000000 55%, #000000 57%, #000000 57%, #000000 59%, #000000 59%, #000000 61%, #000000 61%, #000000 63%, #000000 63%, #000000 65%, #000000 65%, #000000 67%, #000000 67%, #000000 69%, #000000 69%, #000000 71%, #000000 71%, #000000 73%, #000000 73%, #000000 75%, #000000 75%, #000000 77%, #000000 77%, #000000 79%, #000000 79%, #000000 81%, #000000 81%, #000000 83%, #000000 83%, #000000 85%, #000000 85%, #000000 87%, #000000 87%, #000000 89%, #000000 89%, #000000 91%, #000000 91%, #000000 93%, #000000 93%, #000000 95%, #000000 95%, #000000 97%, #000000 97%, #000000 99%, #000000 99%, #000000 100%);color:#e6eef8;padding:24px}
9   .wrap{max-width:980px;margin:0 auto;display:grid;grid-template-columns:1fr 420px;gap:18px}
10  .card{background:linear-gradient(to right, #000000 0%, #000000 49%, #000000 49%, #000000 51%, #000000 51%, #000000 53%, #000000 53%, #000000 55%, #000000 55%, #000000 57%, #000000 57%, #000000 59%, #000000 59%, #000000 61%, #000000 61%, #000000 63%, #000000 63%, #000000 65%, #000000 65%, #000000 67%, #000000 67%, #000000 69%, #000000 69%, #000000 71%, #000000 71%, #000000 73%, #000000 73%, #000000 75%, #000000 75%, #000000 77%, #000000 77%, #000000 79%, #000000 79%, #000000 81%, #000000 81%, #000000 83%, #000000 83%, #000000 85%, #000000 85%, #000000 87%, #000000 87%, #000000 89%, #000000 89%, #000000 91%, #000000 91%, #000000 93%, #000000 93%, #000000 95%, #000000 95%, #000000 97%, #000000 97%, #000000 99%, #000000 99%, #000000 100%);padding:18px;border-radius:10px;box-shadow:0 6px 18px #000000;position:relative}
11  label{display:block;margin-top:8px;color:#9fb0cc;font-size:13px}
12  input[type="text"],input[type="file"]{width:100%;padding:8px;border-radius:8px;background:transparent;border:1px solid #000000;color:inherit}
13  button{margin-top:12px;padding:10px 12px;border-radius:10px;border:0;background:linear-gradient(to right, #000000 0%, #000000 49%, #000000 49%, #000000 51%, #000000 51%, #000000 53%, #000000 53%, #000000 55%, #000000 55%, #000000 57%, #000000 57%, #000000 59%, #000000 59%, #000000 61%, #000000 61%, #000000 63%, #000000 63%, #000000 65%, #000000 65%, #000000 67%, #000000 67%, #000000 69%, #000000 69%, #000000 71%, #000000 71%, #000000 73%, #000000 73%, #000000 75%, #000000 75%, #000000 77%, #000000 77%, #000000 79%, #000000 79%, #000000 81%, #000000 81%, #000000 83%, #000000 83%, #000000 85%, #000000 85%, #000000 87%, #000000 87%, #000000 89%, #000000 89%, #000000 91%, #000000 91%, #000000 93%, #000000 93%, #000000 95%, #000000 95%, #000000 97%, #000000 97%, #000000 99%, #000000 99%, #000000 100%);color:#e6eef8}
14  .log{background:linear-gradient(to right, #000000 0%, #000000 49%, #000000 49%, #000000 51%, #000000 51%, #000000 53%, #000000 53%, #000000 55%, #000000 55%, #000000 57%, #000000 57%, #000000 59%, #000000 59%, #000000 61%, #000000 61%, #000000 63%, #000000 63%, #000000 65%, #000000 65%, #000000 67%, #000000 67%, #000000 69%, #000000 69%, #000000 71%, #000000 71%, #000000 73%, #000000 73%, #000000 75%, #000000 75%, #000000 77%, #000000 77%, #000000 79%, #000000 79%, #000000 81%, #000000 81%, #000000 83%, #000000 83%, #000000 85%, #000000 85%, #000000 87%, #000000 87%, #000000 89%, #000000 89%, #000000 91%, #000000 91%, #000000 93%, #000000 93%, #000000 95%, #000000 95%, #000000 97%, #000000 97%, #000000 99%, #000000 99%, #000000 100%);padding:12px;border-radius:8px;height:420px;overflow:auto;font-family:monospace;font-size:13px;color:#bcd}
15  .muted{color:#93a7bf;font-size:13px}
16  .controls{display:flex;gap:10px;align-items:center;margin-top:8px}
17  .img-preview{max-width:100%;margin-top:10px;border-radius:8px;border:1px solid #000000}
18  footer{grid-column:1/-1;margin-top:12px;color:#93a7bf}
19 </style>
20 </head>
21 <body>
22 <div class="wrap">
23 <div class="card">
24 <h2 style="margin:0 0 8px">LMS Safe Logging – Frontend Demo</h2>
25
26 <form id="frm" onsubmit="return handleSubmit(event)">
27 <label>user_id</label>
28 <input id="user_id" type="text" value="stu123" required/>
29
30 <label>name</label>
31 <input id="name" type="text" value="Akshitha" />
32
33 <label>email</label>
34 <input id="email" type="text" value="akshitha@example.com" />
35
36 <div style="display:flex;gap:10px">
37 <div style="flex:1">
38 <label>course_id</label>
39 <input id="course_id" type="text" value="CS101" />
40 </div>
41 <div style="flex:1">
42 <label>assignment</label>
43 <input id="assignment" type="text" value="hw1" />
44 </div>
45 </div>
46
47 <label>grade</label>
48 <input id="grade" type="text" value="A" />
49
50 <label>file</label>
51 <input id="file" type="file" />
52
53 <div class="controls">
54 <div style="display:flex;align-items:center;gap:8px">
55 <input id="use_server" type="checkbox" /> <span class="muted">Send to real server</span>
56 </div>
57 <div style="margin-left:auto" class="muted">Endpoint: <code id="endpoint">http://127.0.0.1:5000/submit_assignment</code></div>
58 </div>
59
60 <button type="submit">Submit (simulate safe logging)</button>
61 </form>
62
63 <div style="margin-top:12px">
64 <button onclick="downloadSecureErrors()">Download secure_errors.log</button>
65 <button onclick="clearLogs()" style="margin-left:8px">Clear logs</button>
66 </div>
67
68 <div style="margin-top:12px">
69 <strong>Example server-side file (local path)</strong>
70 <div class="muted">/mnt/data/447f4abe-bf11-4970-9dd9-269acc7edcb6.png</div>
71 
72 </div>
73 </div>
74
75 <div class="card">
76 <h3 style="margin:0 0 8px">Log Console (safe)</h3>
77 <div id="log" class="log" aria-live="polite"></div>
78 <div style="margin-top:8px" class="muted">Notes: PIDs are deterministic sha256-8 chars; emails masked; filenames sanitized and hashed. This is client-side demo – enforce server-side
79 </div>
80
81 <footer>
82 Client-side demo only: for production enforce the same transformations server-side (use HMAC with a server secret, anonymize IPs on server, restrict access to secure_errors.log).
83 </footer>
84 </div>
85

```

```

86 <script>
87 // secure_errors.log
88 const secureErrors = []; // array of full stack-trace entries (simulated secure_errors.log)
89 function appendLog(txt){ const ts = new Date().toISOString(); logEl.textContent += `[${ts}] ${txt}\n`; logEl.scrollTop = logEl.scrollHeight; }
90 function clearLogs(){ logEl.textContent=""; appendLog("[console cleared]"); }
91 Complexity is 3 Everything is cool!
92 function downloadSecureErrors(){
93   if(secureErrors.length === 0){ alert('secure_errors.log is empty'); return; }
94   const blob = new Blob([secureErrors.join("\n\n")], {type: 'text/plain;charset=utf-8'});
95   const url = URL.createObjectURL(blob);
96   const a = document.createElement('a'); a.href = url; a.download = 'secure_errors.log'; a.click(); URL.revokeObjectURL(url);
97 }
98 // helper: compute sha256 and return first 8 hex chars
99 Complexity is 5 Everything is cool!
100 async function sha8(input){
101   if(!input) return 'unknown';
102   const enc = new TextEncoder().encode(String(input));
103   const hash = await crypto.subtle.digest('SHA-256', enc);
104   const hex = Array.from(new Uint8Array(hash)).map(b=>b.toString(16).padStart(2,'0')).join('');
105   return hex.slice(0,8);
106 }
107 Complexity is 4 Everything is cool!
108 function redactEmail(email){
109   if(!email) return 'unknown';
110   return email.replace(/^(.+)@(.+)$/, '$1***$2');
111 }
112 Complexity is 5 Everything is cool!
113 function sanitizeFilename(fname){
114   if(!fname) return 'no-file';
115   // basename + simple length limit
116   const base = fname.split(/[\\\/]/).pop();
117   return base.length > 200 ? base.slice(0,200) : base;
118 }
119 // bucket grade only
120 Complexity is 8 it's time to do something...
121 function gradeBucket(g){ if(!g) return 'none'; const v = String(g).trim().toUpperCase(); if(['A+', 'A'].includes(v)) return 'A'; if(['B+', 'B'].includes(v)) return 'B'; return 'C_or_low'; }
122 // When an exception happens we simulate saving full stacktrace (secure) and generate err_id
123 Complexity is 4 Everything is cool!
124 async function saveExceptionSecurely(err, context){
125   const tb = (err && err.stack) ? err.stack : String(err);
126   const payload = `ERR_CONTEXT=${JSON.stringify(context)}\n${tb}\n`;
127   const id = await sha8(payload);
128   secureErrors.push(`err_id=${id}\n${payload}`);
129   return id;
130 }
131 // form handler
132 Complexity is 14 You must be kidding
133 async function handleSubmit(e){
134   e.preventDefault();
135   try {
136     const user_id = document.getElementById('user_id').value;
137     const name = document.getElementById('name').value;
138     const email = document.getElementById('email').value;
139     const course_id = document.getElementById('course_id').value;
140     const assignment = document.getElementById('assignment').value;
141     const grade = document.getElementById('grade').value;
142     const fileInput = document.getElementById('file');
143     const file = fileInput.files[0] || null;
144     // CLIENT-SIDE safe transforms (demo)
145     const pid = await sha8(user_id); // deterministic pseudonym
146     const email_masked = redactEmail(email); // safe email mask
147     // We cannot discover true remote IP in browser; we use 'client' placeholder and hash it
148     const ip_placeholder = 'client-side';
149     const ip_hash = await sha8(ip_placeholder);
150     const file_name = sanitizeFilename(file ? file.name : null);
151     const file_hash = (file_name !== 'no-file') ? await sha8(file_name) : 'no-file';
152     // Minimal safe logging (client-side)
153     appendLog(`submission: pid=${pid} course=${course_id} assignment=${assignment} file=${file_name} file_hash=${file_hash} ip_hash=${ip_hash}`);
154     if(grade) appendLog(`grade_present for pid=${pid} (bucket=${gradeBucket(grade)})`);
155     // Optionally send to server (raw values) - checkbox decides
156     const useServer = document.getElementById('use_server').checked;
157     if(useServer){
158       appendLog(`Sending raw data to server endpoint (ensure HTTPS & server-side enforcement)`);
159       // build FormData
160       const fd = new FormData();
161       fd.append('user_id', user_id);
162       fd.append('name', name);
163       fd.append('email', email);
164       fd.append('course_id', course_id);
165       fd.append('assignment', assignment);
166       fd.append('grade', grade);
167       if(file) fd.append('file', file, file_name);
168       try {
169         const resp = await fetch(document.getElementById('endpoint').textContent.trim(), { method: 'POST', body: fd });
170         const js = await resp.json().catch(()=>null);
171         appendLog(`SERVER RESP: ${resp.status} ${resp.statusText} ${js ? JSON.stringify(js) : ''}`);
172       } catch(sendErr){
173         const err_id = await saveExceptionSecurely(sendErr, { pid, course_id, assignment });
174         appendLog(`Failed to POST to server - err_id=${err_id}`);
175       }
176     } else {
177       appendLog(`Simulated processing complete (no raw data sent).`);
178     }
179   } catch(err){
180     const pid = await sha8(document.getElementById('user_id').value);
181     const err_id = await saveExceptionSecurely(err, { pid });
182     appendLog(`processing_error err_id=${err_id} pid=${pid}`);
183   }
184   return false;
185 }
186 // init
187 clearLogs();
188 appendLog("Frontend safe-logging demo ready. Use 'Send to real server' to POST to backend (optional).");
189 </script>
190 </body>
191 </html>
192

```

OUTPUT:

Task – 1:

The screenshot shows a web browser window with the URL `127.0.0.1:5500/task1.html`. The page title is "Submit Assignment — Safe logging demo". The form contains the following fields and values:

- user id:** stu123
- name:** Sai Varun
- email:** alshilha@example.com
- course id:** CS101
- assignment:** hw1
- grade:** A
- file:** saivarun File (No file chosen)

Below the form, there is a checkbox labeled "Send to real server" with the endpoint `ENDPOINT: http://127.0.0.1:5000/submit_assignment`. A "Submit" button is present. Below the button, it says "Mode: Standalone (simulate)". There are two buttons: "Download logs" and "Clear logs".

At the bottom, there is a section titled "Example screenshot from server path" with the text: "(If your environment exposes files under /tmp/ then this image will show)".

On the right side, there is a "Log Console" panel showing the following log output:

```
[console cleared] hola[app] Safe logging
frontend ready, use "Send to real server" to
post to flask at
http://127.0.0.1:5000/submit_assignment\n
```

Below the log console, it says: "These logs are safe (user id is pseudonymized, email is redacted, full path not logged)".

Task – 2:

The screenshot shows a web browser window with the URL `127.0.0.1:5500/task2.html`. The page title is "LMS Safe Logging — Frontend Demo". The form contains the following fields and values:

- user id:** Sai Varun
- name:** Sai Varun
- email:** alshilha@example.com
- course id:** CS101
- assignment:** hw1
- grade:** A
- file:** Choose File (No file chosen)

Below the form, there is a checkbox labeled "Send to real server" with the endpoint `Endpoint: http://127.0.0.1:5000/submit_assignment`. A button labeled "Submit (simulate safe logging)" is present. Below the button, there are two buttons: "Download secure_errors.log" and "Clear logs".

At the bottom, there is a section titled "Example server-side file (local path)" with the text: `/tmp/data/84774a6e-b411-4970-9d30-108acc7a33b6.png`.

On the right side, there is a "Log Console (safe)" panel showing the following log output:

```
[2023-11-24T08:21:23.029Z] [console cleared]
[2023-11-24T08:21:23.071Z] frontend safe-logging
demo ready, use "Send to real server" to POST to
backend (optional).\n
```

Below the log console, there is a note: "Notes: PIDs are deterministic sha256-6 chars; emails masked; filenames sanitized and hashed. This is client-side demo — enforce server-side as well."

At the bottom of the page, there is a disclaimer: "Client-side demo only. for production enforce the same transformations server-side (use HMAC with a server secret, anonymize IPs on server, restrict access to secure_errors.log)".

OBSERVATION:

1. The HTML/CSS/JS version applies the same privacy-safe rules as the Flask code, including pseudonymized user IDs, masked emails, sanitized filenames, and hashed file identifiers.

2. No raw PII, session tokens, or full headers are logged in the browser console, reducing accidental exposure during client-side logging.
3. Errors generate a short err_id while full stack traces are stored separately in a simulated secure log, mirroring server-side secure error handling.
4. This frontend demo reinforces privacy-by-design, but real enforcement must still happen on the backend for true security.

Q2: PROMPT:

Task – 1:

“Find algorithmic bias in this course recommendation code:

[PASTE CODE]

Identify biases related to demographics, historical data, or feedback loops.”

Task – 2:

“Refactor this code to remove bias and add fairness constraints:

[PASTE CODE]

*Implement demographic parity, remove discriminatory features,
and add bias monitoring metrics.”*

CODE GENERATED:

Task – 1:

```
t3.py X
t3.py > recommend_courses

1
2 from typing import Dict, List, Optional
3 import math
4 import statistics
5
6 EXAMPLE_FILE_URL = "/mnt/data/447f4abe-bf11-4970-9dd9-269acc7edcb6.png"
7
8 # --- Helper utilities -----
9
10 def _normalize(value: Optional[float], Lo: float = 0.0, hi: float = 100.0) -> float:
11     """Normalize numeric values into [0,1]. Returns 0.0 for missing/invalid."""
12     try:
13         v = float(value)
14     except (TypeError, ValueError):
15         return 0.0
16     if math.isnan(v):
17         return 0.0
18     return max(0.0, min(1.0, (v - Lo) / (hi - Lo)))
19
20
21 def _score_for_advanced(past_grade_avg: Optional[float], has_relevant_skills: bool) -> float:
22
23     grade_norm = _normalize(past_grade_avg, 0, 100) # 0..1
24     skill_bonus = 0.2 if has_relevant_skills else 0.0
25     # soft logistic-style curve to avoid a hard cut
26     score = (0.6 * grade_norm) + skill_bonus
27     return score
28
29
30 # --- Main recommendation function -----
31
32 def recommend_courses(user: Dict, *, top_k: int = 5, sponsored_cap: int = 1) -> List[str]:
33
34     # read catalog from json file
35     with open('catalog.json', 'r') as f:
36         catalog = json.load(f)
```

```

32 def recommend_courses(user: Dict, *, top_k: int = 5, sponsored_cap: int = 1) -> List[str]:
33
34
35     # Read safe signals (do not rely on gender/age/zip)
36     past_grade_avg = user.get('past_grade_avg')
37     degree = (user.get('degree_level') or '').lower()
38     prefers_part_time = bool(user.get('prefers_part_time'))
39     has_relevant_skills = bool(user.get('has_relevant_skills'))
40     ability_to_pay = _normalize(user.get('ability_to_pay', 0.0), 0.0, 1.0)
41     referral = user.get('referral_code')
42
43     # Candidate catalog with simple metadata
44     catalog = {
45         'part_time_fundamentals': {'part_time': True, 'level': 'foundation', 'paid': False},
46         'time_friendly_courses': {'part_time': True, 'level': 'foundation', 'paid': False},
47         'advanced_machine_learning': {'part_time': False, 'level': 'advanced', 'paid': True},
48         'data_science_project': {'part_time': False, 'level': 'advanced', 'paid': True},
49         'premium_ai_program': {'part_time': False, 'level': 'advanced', 'paid': True},
50         'executive_leadership': {'part_time': False, 'level': 'advanced', 'paid': True},
51         'placement_support': {'part_time': False, 'level': 'placement', 'paid': False},
52         'career_boost_program': {'part_time': False, 'level': 'intermediate', 'paid': False},
53         'foundation_program': {'part_time': True, 'level': 'foundation', 'paid': False},
54         'sponsored_onboarding': {'part_time': False, 'level': 'foundation', 'paid': True, 'sponsored': True}
55     }
56
57     scores = {}
58
59     # 1) Part-time preference scoring
60     for cid, meta in catalog.items():
61         score = 0.0
62         # boost if part-time and user prefers it
63         if prefers_part_time and meta.get('part_time'):
64             score += 0.2
65         # degree match: prefer programs aligned to degree level
66         if degree in ('bachelors', 'masters') and meta.get('level') in ('intermediate', 'advanced'):
67             score += 0.15
68         if degree not in ('bachelors', 'masters') and meta.get('level') == 'foundation':
69             score += 0.12
70
71     # advanced STEM scoring
72     adv_score = _score_for_advanced(past_grade_avg, has_relevant_skills)
73     if meta.get('level') == 'advanced':
74         score += 0.5 * adv_score
75
76     # ability to pay slightly increases score for paid programs (but will not exclude non-paying options)
77     if meta.get('paid'):
78         score += 0.25 * ability_to_pay
79
80     # small randomization/stability term can be added in production to diversify recommendations
81     scores[cid] = score
82
83     # 2) Convert scores to ranked list
84     ranked = sorted(scores.items(), key=lambda x: x[1], reverse=True)
85     recommended = [cid for cid, _ in ranked if not catalog[cid].get('sponsored')]
86
87     # 3) Include sponsored slot(s) if referral exists, but cap influence and mark clearly
88     sponsored_items = []
89     if referral:
90         # map referral to a sponsored offering; in production this mapping should be auditable
91         sponsored_items = ['sponsored_onboarding']
92         sponsored_items = sponsored_items[:sponsored_cap]
93
94     # 4) Post-processing: ensure placement support is available if user is seeking employment
95     # Avoid filtering out placements by age or other protected attributes
96     # Instead, add placement support for users with mid-high scores or explicit request
97     wants_placement = bool(user.get('wants_placement'))
98     if wants_placement:
99         # push placement_support up the list without removing others
100         recommended = ['placement_support'] + recommended
101
102     # 5) Deduplicate while preserving order
103     final = []

```

```

104     for cid in (sponsored_items + recommended):
105         if cid not in final:
106             final.append(cid)
107     # Limit to top-k
108     return final[:top_k]
109
110 # --- Fairness utilities -----
111
112 def recommendation_parity_rate(users: List[Dict], group_fn, course_id: str) -> Dict[str, float]:
113     """Compute the fraction of users in each group (group_fn(user) -> group_key)
114     who receive course_id in their top-1 recommendation. Useful for parity checks.
115     Returns a map group_key -> rate (0..1).
116     """
117     groups = {}
118     for u in users:
119         g = group_fn(u)
120         top = recommend_courses(u, top_k=1)
121         groups.setdefault(g, []).append(1 if course_id in top else 0)
122     rates = {g: (sum(vals) / len(vals)) if len(vals) else 0.0 for g, vals in groups.items()}
123     return rates
124
125 # --- Simple command-line demo / smoke test -----
126
127 if __name__ == '__main__':
128     sample_users = [
129         {'user_id': 'u1', 'past_grade_avg': 85, 'degree_level': 'bachelors', 'prefers_part_time': True, 'has_relevant_skills': True, 'ability_to_pay': 0.9, 'referral_code': 'R1', 'demographic_group': 'A' },
130         {'user_id': 'u2', 'past_grade_avg': 75, 'degree_level': 'none', 'prefers_part_time': True, 'has_relevant_skills': False, 'ability_to_pay': 0.1, 'demographic_group': 'B' },
131         {'user_id': 'u3', 'past_grade_avg': 82, 'degree_level': 'masters', 'prefers_part_time': False, 'has_relevant_skills': True, 'ability_to_pay': 0.5, 'demographic_group': 'A' },
132         {'user_id': 'u4', 'past_grade_avg': 60, 'degree_level': 'none', 'prefers_part_time': False, 'has_relevant_skills': False, 'ability_to_pay': 0.0, 'demographic_group': 'B' },
133     ]
134
135     for u in sample_users:
136         print(u['user_id'], '->', recommend_courses(u, top_k=4))
137
138     # Quick parity check for 'placement_support' by demographic_group
139     rates = recommendation_parity_rate(sample_users, lambda u: u.get('demographic_group', 'unknown'), 'placement_support')
140     print("\nParity rates for placement_support by demographic_group:", rates)
141
142

```

Task – 2:

```

1 # recommendation_rules_fair.py
2 # Ctrl+L to chat, Ctrl+K to generate
3
4 from typing import Dict, List, Callable, Tuple
5 import math
6
7 EXAMPLE_FILE_URL = "/mnt/data/447f4abe-bf11-4970-9dd9-269acc7edcb6.png"
8
9 # ----- original scoring utilities (kept from previous refactor) -----
10 def _normalize(value, lo=0.0, hi=100.0):
11     try:
12         v = float(value)
13     except (TypeError, ValueError):
14         return 0.0
15     if math.isnan(v):
16         return 0.0
17     return max(0.0, min(1.0, (v - lo) / (hi - lo)))
18
19 def _score_for_advanced(past_grade_avg, has_relevant_skills):
20     grade_norm = _normalize(past_grade_avg, 0, 100)
21     skill_bonus = 0.2 if has_relevant_skills else 0.0
22     return (0.6 * grade_norm) + skill_bonus
23
24 # ----- catalog and base recommender (similar to prior) -----
25 CATALOG = {
26     'part_time_fundamentals': {'part_time': True, 'level': 'foundation', 'paid': False},
27     'time_friendly_courses': {'part_time': True, 'level': 'foundation', 'paid': False},
28     'advanced_machine_learning': {'part_time': False, 'level': 'advanced', 'paid': True},
29     'data_science_project': {'part_time': False, 'level': 'advanced', 'paid': True},
30     'premium_ai_program': {'part_time': False, 'level': 'advanced', 'paid': True},
31     'executive_leadership': {'part_time': False, 'level': 'advanced', 'paid': True},
32     'placement_support': {'part_time': False, 'level': 'placement', 'paid': False},
33     'career_boost_program': {'part_time': False, 'level': 'intermediate', 'paid': False},
34     'foundation_program': {'part_time': True, 'level': 'foundation', 'paid': False},
35     'sponsored_onboarding': {'part_time': False, 'level': 'foundation', 'paid': True, 'sponsored': True}
36 }
37
38 def base_scores_for_user(user: Dict) -> Dict[str, float]:
39     past_grade_avg = user.get('past_grade_avg')
40     degree = (user.get('degree_level') or '').lower()
41     prefers_part_time = bool(user.get('prefers_part_time'))
42     has_relevant_skills = bool(user.get('has_relevant_skills'))
43     ability_to_pay = _normalize(user.get('ability_to_pay', 0.0), 0.0, 1.0)
44
45     scores = {}
46     adv_score = _score_for_advanced(past_grade_avg, has_relevant_skills)
47     for cid, meta in CATALOG.items():
48         score = 0.0
49         if prefers_part_time and meta.get('part_time'):
50             score += 0.2

```

```

51         if degree in ('bachelors', 'masters') and meta.get('level') in ('intermediate', 'advanced'):
52             score += 0.15
53         if degree not in ('bachelors', 'masters') and meta.get('level') == 'foundation':
54             score += 0.12
55         if meta.get('level') == 'advanced':
56             score += 0.5 * adv_score
57         if meta.get('paid'):
58             score += 0.25 * ability_to_pay
59         scores[cid] = score
60     return scores
61
62 def recommend_ranked(user: Dict, top_k: int = 5) -> List[Tuple[str, float]]:
63     """Return ranked list of (course_id, score) excluding sponsored items from top ranking decision."""
64     scores = base_scores_for_user(user)
65     ranked = sorted(scores.items(), key=lambda x: x[1], reverse=True)
66     # keep sponsored separate for controlled insertion
67     return ranked
68
69 # ----- Fairness reranker -----
70 def compute_group_exposure(top1_list: List[Tuple[str, str]], group_fn: Callable[[Dict], str]) -> Dict[str, float]:
71     """
72     top1_list: list of (user_id, top1_course)
73     group_fn: maps user object (or user_id->group) - in batch flow we map via provided map
74     Returns group -> exposure_rate (top1 fraction)
75     """
76     counts = {}
77     totals = {}
78     for user_id, group, top1 in top1_list:
79         totals.setdefault(group, 0)
80         counts.setdefault(group, 0)
81         totals[group] += 1
82         if top1:
83             counts[group] += 1 if top1 else 0
84     rates = {g: (counts[g] / totals[g]) if totals[g] else 0.0 for g in totals}
85     return rates
86
87 def fairness_rerank_batch(
88     users: List[Dict],
89     group_fn: Callable[[Dict], str],
90     target_course: str = 'placement_support',
91     epsilon: float = 0.05,
92     top_k: int = 5,
93     max_boost: float = 1.0
94 ) -> Tuple[Dict[str, List[str]], Dict]:
95     """
96     Batch pipeline:
97     1. Compute base top-1 per user.
98     2. Compute group exposure rates for target_course.
99     3. While max_rate - min_rate > epsilon:
100         - identify under-exposed groups
101         - for users in under-exposed groups, boost the score for target_course by a small increment
102         - recompute top-1 and group rates
103         - stop when parity reached or budget exhausted
104     Returns:
105         - map user_id -> final recommended top_k list
106         - diagnostics dict (rates history, boosts applied)
107     """
108     # prepare per-user scores and metadata
109     user_scores = {}
110     user_map = {}
111     for u in users:
112         uid = u.get('user_id') or f"u_{id(u)}"
113         user_map[uid] = u
114         user_scores[uid] = {cid: score for cid, score in base_scores_for_user(u).items()}
115
116     diagnostics = {'history': []}
117     # compute an initial top1 list
118     def current_top1_list():
119         result = []
120         for uid, scores in user_scores.items():
121             top1 = max(scores.items(), key=lambda x: x[1])[0]
122             grp = group_fn(user_map[uid])
123             result.append((uid, grp, top1))
124         return result
125
126     top1 = current_top1_list()
127     rates = compute_group_exposure(top1, group_fn)
128     diagnostics['history'].append({'rates': rates.copy()})
129
130     budget = max_boost # total boost budget per user group in aggregate (simple control)
131     step = 0.2 # incremental boost per iteration
132     iteration = 0
133     # iterate until parity within epsilon or budget exhausted or iterations cap
134     while True:
135         iteration += 1
136         if iteration > 20:
137             diagnostics['note'] = 'hit iteration cap'
138             break
139         values = list(float(rates.values())) if rates else [0.0]
140         max_rate = max(values)
141         min_rate = min(values)
142         if max_rate - min_rate <= epsilon:
143             diagnostics['note'] = 'parity achieved'
144             break
145         # identify under-exposed groups (those with rate <= min_rate + tiny)
146         under_groups = [g for g, r in rates.items() if r < max_rate - epsilon/2]

```

```

147 def fairness_rerank_batch:
148     if not under_groups:
149         diagnostics['note'] = 'no clear under-exposed groups'
150         break
151
152     # apply boost to target_course for users in under-exposed groups
153     boosted = 0
154     for uid, u in user_map.items():
155         grp = group_fn(u)
156         if grp in under_groups:
157             # boost bounded by budget per user (reduce global budget accordingly)
158             increment = min(step, budget)
159             user_scores[uid][target_course] = user_scores[uid].get(target_course, 0.0) + increment
160             budget -= increment
161             boosted += 1
162             if budget <= 0:
163                 break
164         diagnostics['history'].append({'iteration': iteration, 'boosted_users': boosted, 'remaining_budget': budget})
165     # recompute top1 and rates
166     top1 = current_top1_list()
167     rates = compute_group_exposure(top1, group_fn)
168     diagnostics['history'].append({'rates': rates.copy()})
169     if budget <= 0:
170         diagnostics['note'] = 'budget exhausted'
171         break
172
173     # Build final top-k per user, enforcing sponsored cap and labeling
174     final_recs = {}
175     for uid, scores in user_scores.items():
176         # sort by final score
177         ranked = sorted(scores.items(), key=lambda x: x[1], reverse=True)
178         # ensure sponsored items are capped: don't allow sponsored_onboarding to occupy >1 slot at front
179         sponsored_count = 0
180         for cid, sc in ranked:
181             if CATALOG.get(cid, []).get('sponsored'):
182                 if sponsored_count < 1:
183                     recs.append(cid)
184                     sponsored_count += 1
185                 else:
186                     continue
187             else:
188                 recs.append(cid)
189                 if len(recs) >= top_k:
190                     break
191         final_recs[uid] = recs
192
193     return final_recs, diagnostics
194
195 # ----- small demo / smoke test -----
196 if __name__ == '__main__':
197     # sample users with demographic_group as an explicit safe attribute used only for fairness checks
198     users = [
199         {'user_id': 'u1', 'past_grade_avg': 85, 'degree_level': 'bachelors', 'prefers_part_time': True, 'has_relevant_skills': True, 'ability_to_pay': 0.9, 'referral_code': 'R1', 'demographic_group': 'A'},
200         {'user_id': 'u2', 'past_grade_avg': 75, 'degree_level': 'none', 'prefers_part_time': True, 'has_relevant_skills': False, 'ability_to_pay': 0.1, 'demographic_group': 'B'},
201         {'user_id': 'u3', 'past_grade_avg': 82, 'degree_level': 'masters', 'prefers_part_time': False, 'has_relevant_skills': True, 'ability_to_pay': 0.5, 'demographic_group': 'A'},
202         {'user_id': 'u4', 'past_grade_avg': 60, 'degree_level': 'none', 'prefers_part_time': False, 'has_relevant_skills': False, 'ability_to_pay': 0.8, 'demographic_group': 'B'},
203     ]
204
205     # group function: read from safe attribute demographic_group
206     group_fn = lambda u: u.get('demographic_group', 'unknown')
207
208     final_recs, diag = fairness_rerank_batch(users, group_fn, target_course='placement_support', epsilon=0.05, top_k=4)
209     print("Final recommendations (top-4) per user:")
210     for uid, recs in final_recs.items():
211         print(uid, "-->", recs)
212     print("\nDiagnostics summary:")
213     print(diag)
214

```

OUTPUT:

Task – 1:

```
PS C:\Users\adepu\OneDrive\Desktop\AIAC END> C:/Users/adepu/AppData/Local/Programs/Python/Python313/python.exe Q2_Task1_bias_detection.py
=====
BIAS DETECTION IN COURSE RECOMMENDATION ENGINE
=====

TEST CASE 1: Gender Bias Detection
-----
[BIAS] Limiting to beginner courses due to age: 20
[BIAS] Limiting to beginner courses due to age: 20

Female student (identical GPA/age/location) recommended: ['ENG101', 'ART101', 'PSY101', 'CS101', 'BUS301']
Male student (identical GPA/age/location) recommended: ['CS101', 'PSY101', 'BUS301']

🚩 BIAS DETECTED: Gender-based stereotyping in recommendations

TEST CASE 2: Age Discrimination
-----
[BIAS] Limiting to beginner courses due to age: 19
[BIAS] Removed advanced courses due to age: 40

Young student (19, GPA 3.9) recommended: ['CS101', 'PSY101', 'BUS301']
Older student (40, GPA 3.9) recommended: ['CS101', 'CS201', 'PSY101', 'BUS301']

🚩 BIAS DETECTED: Age-based discrimination limiting opportunities

TEST CASE 3: Socioeconomic Bias
-----
[BIAS] Removed expensive courses for low-income student

Low-income student (GPA 3.7) recommended: ['CS101', 'CS201', 'MATH301', 'PSY101', 'BUS301']
High-income student (GPA 3.7) recommended: ['CS101', 'CS201', 'MATH301', 'MECH401', 'BUS301']

🚩 BIAS DETECTED: Income-based discrimination in course access

TEST CASE 4: Geographic Bias
-----
[BIAS] Limiting course difficulty based on location: rural

Rural student (GPA 3.8) recommended: ['ENG101', 'ART101', 'PSY101', 'CS101', 'BUS301']
Urban student (GPA 3.8) recommended: ['ENG101', 'ART101', 'NURS201', 'PSY101', 'EDU201']

🚩 BIAS DETECTED: Location-based assumptions about capability
```

```

PS C:\Users\adepu\OneDrive\Desktop\AIAC END> C:/Users/adepu/AppData/Local/Programs/Python/Python313/python.exe Q2_Task1_bias_detection.py

=====
AI ANALYSIS: COMPREHENSIVE BIAS DETECTION REPORT
=====

BIAS CATEGORIES IDENTIFIED:

1. **GENDER BIAS** (Severity: HIGH)
  - Stereotypical course recommendations based on gender
  - Different courses recommended for identical qualifications
  - Penalty scoring for non-stereotypical choices
  - Violates: Title IX, Equal Opportunity Act

2. **AGE DISCRIMINATION** (Severity: HIGH)
  - Older students denied advanced course opportunities
  - Younger students artificially limited to beginner courses
  - Assumptions about capability based on age alone
  - Violates: Age Discrimination Act

3. **SOCIOECONOMIC BIAS** (Severity: HIGH)
  - Low-income students denied expensive/advanced courses
  - Assumptions about completion based on income
  - Creates educational inequality and limits social mobility
  - Violates: Equal Opportunity principles

4. **GEOGRAPHIC BIAS** (Severity: MEDIUM)
  - Rural students assumed less prepared
  - Location-based course difficulty restrictions
  - Perpetuates urban-rural education gap

5. **HISTORICAL DATA BIAS** (Severity: HIGH)
  - Using biased past data to make future predictions
  - Perpetuates existing discrimination patterns
  - Self-fulfilling prophecy effect

6. **CONFIRMATION BIAS** (Severity: MEDIUM)
  - Over-recommending based on single past choice
  - Creates filter bubbles and limits exploration
  - Reduces educational diversity

7. **AVAILABILITY BIAS** (Severity: MEDIUM)
  - Popular courses over-recommended regardless of fit
  - Individual needs ignored for aggregate trends

8. **FIRST-GENERATION STUDENT BIAS** (Severity: HIGH)
  - Lower expectations for first-generation students
  - Reduced opportunities based on family background
  - Perpetuates educational inequality

```

Task – 2:


```
PS C:\Users\adept\OneDrive\Desktop\AIAC END> C:/Users/adept/AppData/Local/Programs/Python/Python313/python.exe Q2_Task2_fair_recommendations.py
=====

TEST CASE 1: Fairness Across Demographics (Same Interests)
-----

Student A (Female, rural, low-income) - Top 3 Recommendations:
  1. Intro to Computer Science (Score: 96.7)
    Reason: Matches your interest in STEM; Good match for your skills (programming, problem_solving); Strongly aligns with your career goals; No prerequisites required; Matches hands-on learning style
  2. Data Structures (Score: 88.3)
    Reason: Matches your interest in STEM; Builds on skills: programming; Strongly aligns with your career goals; Prerequisites met; Matches hands-on learning style
  3. Advanced Calculus (Score: 73.3)
    Reason: Matches your interest in STEM; Builds on skills: problem_solving; Strongly aligns with your career goals; Prerequisites needed: MATH201; Matches hands-on learning style

Student B (Male, urban, high-income) - Top 3 Recommendations:
  1. Intro to Computer Science (Score: 96.7)
    Reason: Matches your interest in STEM; Good match for your skills (programming, problem_solving); Strongly aligns with your career goals; No prerequisites required; Matches hands-on learning style
  2. Data Structures (Score: 88.3)
    Reason: Matches your interest in STEM; Builds on skills: programming; Strongly aligns with your career goals; Prerequisites met; Matches hands-on learning style
  3. Advanced Calculus (Score: 73.3)
    Reason: Matches your interest in STEM; Builds on skills: problem_solving; Strongly aligns with your career goals; Prerequisites needed: MATH201; Matches hands-on learning style

✅ FAIRNESS ACHIEVED: Identical recommendations despite demographic differences

TEST CASE 2: Age-Independent Recommendations
-----

Young student (age 19): Mechanical Engineering (Score: 81.7)
Older student (age 45): Mechanical Engineering (Score: 81.7)

✅ FAIRNESS ACHIEVED: Age does not affect recommendations

TEST CASE 3: Fairness Audit Across Groups
-----

Fairness Audit Results:

Overall Fairness Score: 86.3/100

Difficulty Distribution Across Groups:
  female: {'beginner': 2, 'intermediate': 1, 'advanced': 2}
  male: {'beginner': 2, 'intermediate': 1, 'advanced': 2}

PS C:\Users\adept\OneDrive\Desktop\AIAC END> C:/Users/adept/AppData/Local/Programs/Python/Python313/python.exe Q2_Task2_fair_recommendations.py
Overall Fairness Score: 86.3/100

Difficulty Distribution Across Groups:
  female: {'beginner': 2, 'intermediate': 1, 'advanced': 2}
  male: {'beginner': 2, 'intermediate': 1, 'advanced': 2}
  young: {'advanced': 2, 'beginner': 2, 'intermediate': 1}
  older: {'advanced': 2, 'beginner': 2, 'intermediate': 1}

Equal Opportunity Metrics:
  female: 40.0% advanced courses
  male: 40.0% advanced courses
  young: 40.0% advanced courses
  older: 40.0% advanced courses

⚠️ FAIRNESS CONCERN: Score below 90 indicates potential disparities

TEST CASE 4: Transparent Explanations
-----

Course: Data Structures (CS201)
Recommendation Score: 88.3/100

Explanation:
Matches your interest in STEM; Builds on skills: programming; Strongly aligns with your career goals; Prerequisites met; Matches hands-on learning style

This recommendation is based solely on your:
- Academic interests: STEM, technology
- Skills: programming, problem_solving
- Career goals: software_engineer
- Completed courses: CS101

NOT based on: gender, age, race, ethnicity, income, location, or other protected characteristics.

✅ TRANSPARENCY: Students receive clear explanations for recommendations

=====
FAIRNESS IMPROVEMENTS SUMMARY
=====

BEFORE (Task 1 - Biased): | AFTER (Task 2 - Fair):
-----|-----
X Gender stereotypes in recommendations | ✓ Gender-neutral scoring
X Age discrimination | ✓ Age-independent recommendations
X Socioeconomic bias | ✓ Income-independent access
X Geographic assumptions | ✓ Location-neutral recommendations
X Historical data perpetuates bias | ✓ Focus on individual merit
```


OBSERVATION:

A batch-level *post-hoc reranker* was added to enforce exposure parity for a target course (e.g., placement_support) by boosting scores for users in under-exposed groups until parity (within epsilon) is reached.

- The approach is deterministic, auditable, and keeps sponsored items capped so commercial slots cannot fully override fairness adjustments.
- Diagnostics record per-iteration rate changes and boosts, enabling monitoring and rollback if undesired side effects appear.
- This is a light-weight, production-friendly method — for stronger guarantees use optimization solvers or causal approaches and always run server-side with logging/monitoring.